# Accessible Skimming: Faster Screen Reading of Web Pages

**Faisal Ahmed** **Yevgen Borodin** **Andrii Soviak**
**Muhammad Asiful Islam** **I.V. Ramakrishnan** **Terri Hedgpeth**[†]
Stony Brook University, Computer Science Department, Stony Brook, NY 11790-4400
[†]Disability Resource Center, Arizona State University, Tempe, AZ 85287-3202
{faiahmed, borodin, asoviak, maislam, ram}@cs.stonybrook.edu, [†]terrih@asu.edu

**ABSTRACT**
In our information-driven web-based society, we are all gradually falling "victims" to *information overload* [5].However, while sighted people are finding ways to sift through information faster, Internet users who are blind are experiencing an even greater information overload. These people access computers and Internet using screen-reader software, which reads the information on a computer screen sequentially using computer-generated speech. While sighted people can learn how to quickly glance over the headlines and news articles online to get the gist of information, people who are blind have to use keyboard shortcuts to listen through the content narrated by a serial audio interface. This interface does not give them an opportunity to know what content to skip and what to listen to. So, they either listen to all of the content or listen to the first part of each sentence or paragraph before they skip to the next one. In this paper, we propose an automated approach to facilitate *non-visual skimming* of web pages. We describe the underlying algorithm, outline a non-visual skimming interface, and report on the results of automated experiments, as well as on our user study with 23 screen-reader users. The results of the experiments suggest that we have been moderately successful in designing a viable algorithm for automatic summarization that could be used for non-visual skimming. In our user studies, we confirmed that people who are blind could read and search through online articles faster and were able to understand and remember most of what they have read with our skimming system. Finally, all 23 participants expressed genuine interest in using non-visual skimming in the future.

**Author Keywords**
Skimming; Blind; Speed Reading; Screen Reader; Summarization; Accessibility; Assistive Technology.

**ACM Classification Keywords**
H.5.2 [Information Interfaces and Presentation]: User Interfaces; H.5.4 [Information Interfaces and Presentation]: Hypertext/Hypermedia -*navigation.*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
*UIST '12,* October 7–10, 2012, Cambridge, Massachusetts, USA.
Copyright 2012 ACM 978-1-4503-1580-7/12/10...$15.00.

**INTRODUCTION**
When sighted users look at a random web page, their quick eye movements, called saccades, help them get a quick overview of the page and find what they need. The structure and formatting of the web page further helps them find the information relevant to their goals. In most web pages, this still leaves them a lot of content to read through, imposing a *heavy cognitive load*. But again, saccades help them speed up reading and reduce the cognitive load. The process of quickly glancing over the textual content in order to pick up the gist of the content is called *Skimming*.

*Skimming* [24, 28] is a speed-reading technique that allows sighted readers to get a general idea of the content and sift through information very quickly, without reading the entire content and, most importantly, without sacrificing basic understanding of the content. Skimming is also very useful while looking for specific information, especially, when keyword searching is not possible. Unfortunately, people who are blind cannot skim through text due to the way screen-reading software works. Screen-readers read all text in a window from start to finish. Though screen-reader users can move through information by character, word, line, paragraph, section, page, etc, these functions are not equivalent to the ability to skim through information [6].

Screen readers [21, 29, 35] are assistive technology software that narrate the content of the screen using text-to-speech and allow users to navigate through the content marked up by HTML tags for headings, paragraphs, links, buttons, etc. Because of the low bandwidth of a serial audio interface and the fact that one has to hear the information before deciding if it is relevant, people who are blind spend considerably more time identifying the information they need [8]. A lot of research [2, 7, 9, 10, 19, 26] has been done to accelerate this process by helping find the main content in web pages. However, little has been done to develop an effective method for skimming, so that blind people could process information faster. For example, having found the beginning of an article in a web page, screen-reader users cannot just scan through the article.

The JAWS screen-reader has a naive implementation of "skimming" [22] that allows users to read the first line or the first sentence of each paragraph. This still does not offer a truly functional tool for skimming, and does not follow the same skimming protocol that sighted folks employ. An earlier experiment with 20 people [1] who are blind showed

that skipping sentences with shortcuts and increasing the speech rate are the *only* strategies screen-reader users use to skim through web pages. However, these strategies are no match to visual skimming, in which quick saccades of the eye allow sighted people to pick out snippets of information such as names, dates, phrases, which allow sighted people to get the gist of the content or find the exact information they are looking for much faster.

The earlier study [1] reported the usability of non-visual skimming in listening-and-comprehension and in search scenarios. To simulate skimming, they used several variations of gold-standard (human-generated) summaries. These experiments demonstrated that skimming enabled people who are blind to read faster while maintaining high level of comprehension and retention.

In this paper, we review the related work on summarization and explain why they are not suitable to support non-visual skimming. We propose a new algorithm that can produce summaries for non-visual skimming of *textual* content and report on the experiments over a dataset of human-generated summaries. Finally, we present the results of the user study with 23 blind people to confirm the usability of the approach.

## RELATED WORK

### Overview of Skimming

People typically read for enjoyment, to search for information, to complete a task, or to explore / review text [24]. Whatever the specific purpose of reading may be, people are often faced with large volumes of information that they need to process. *Cognitive overload* [33]is a fundamental problem that arises while processing a large volume of information in web pages; this is especially true for people with vision impairments who have to listen through most of web content in a sequential manner. Hence, a tool for generating semantic driven summaries that are comparable to the way sighted people skim through information would give screen-reader users, the ability to sift through information reducing the cognitive load.

To reduce the cognitive load, sighted people employ a number of different speed-reading techniques (a.k.a. skimming or scanning). Some people define scanning as looking for specific information in content and skimming as getting the content's gist, others call both of them skimming. Our method covers for both of these fast-reading techniques, and we refer to them collectively as "skimming". While skimming, sighted people quickly scan line by line picking out words and phrases that seem to stand out. This enables sighted people to read several times faster and still get the gist of the content. Skimming can also be very useful when looking for specific information that could not be searched for otherwise. Unfortunately, assistive technology does not provide people who are blind the ability to efficiently skim through information in a manner that is comparable to that of sighted people.

To simulate the skimming process, one could try to use eye tracking devices to figure out what words and phrases are chosen by the human eye. However, different people develop different ad hoc approaches to skimming, and there is no guarantee that the words that "catch the eye" are the ones that carry the most meaning. An alternative approach is to look specifically for the words and phrases that help preserve the meaning of the text. This could be accomplished by using automated summarization.

Automated summarization approaches use computer algorithms [31] to condense textual content while preserving its gist. Broadly, the approaches could be categorized into two main types: summarization by extraction and summarization by abstraction. In short, extraction deals with picking out the most salient sentences or phrases from the original text [3, 14, 15], and abstraction is compressing sentences [4, 13], which involves paraphrasing the content, for example, by automatically filling out a template [20].

Visual skimming is akin to the extractive summarization, because when people scan the text they take it as it is. Interviews with 20 screen-reader users [1] also suggested that the extractive summarization approach that preserves the original content is the most suitable technique for enabling non-visual skimming. We next review the existing extractive summarization approaches and explain why they are not adequate for non-visual skimming of web pages.

### Extractive Summarization Approaches

#### *Summarization by Sentence Extraction*

The majority of the existing extractive summarization approaches are doing sentence extraction [4, 12, 15, 17]. Such approaches typically analyze the entire text and examine the relationship between sentences to find the ones that have the most meaning or are very different from the rest of the text. JAWS screen reader [21] has a "skimming" feature [22] allowing users to read the 1st sentence or a line of each paragraph of text.

While sentence extraction can help get an overview of text, this approach cannot enable skimming, in which the reader scans through all or most of the sentences picking out the most salient words and phrases. Sentence extraction leaves out most of the original content, which makes it unusable for scanning for information that cannot be found without reading the text, e.g., when keyword search is not possible due to unknown keywords. Sentence extraction assumes coherent text and will not work in web pages, where there is unrelated content, e.g., in front pages of news sites. Therefore, keyword extraction approaches appear to be the only ones suitable for non-visual skimming of web pages.

#### *Summarization by Keyword Extraction*

Based on a preliminary study [1] with people who are blind, the authors came up with important criteria for the structure of the summary that can be used for skimming. The study shows that, the subjects were able to understand the gist of the content much better when they were presented word combinations as opposed to separate words. In addition, the subjects did not want information to be skipped or presented out of order. Based on these findings, we can conclude that

the desired approach should be able to summarize each sentence in a given document, by extracting meaningful words and word combinations from every sentence, while preserving the original order of the words. To be specific it should summarize each sentence separately, without considering any global features (i.e. word frequency) of the entire document.

We next examine relevant extractive summarization approaches to determine if they fit to our criteria.

In [36], the summary is constructed by extracting word trigrams that frequently occur in the entire document. This approach does not pick words from each sentence and needs long documents.

Marina Litvak presents a keyword extraction approach based on the analysis of a whole document [25]. The approach builds a tree where nodes represent words and edges represent order relationships. Higher weights are given to the words that have higher number of incoming edges, number of outgoing edges, frequency and tf-idf score. Finally, individual keywords that have the highest weights are extracted into the summary.

In [18], the authors build separate graphs for each sentence and weigh each vertex based on the word and structural dependencies within a sentence, as well as word frequency in the entire document. The summary is generated by selection of top ranked words. The compression of the summary depends on the predefined percent value.

An interesting relation-driven summarization algorithm is proposed in [20]. Authors describe three main stages: 1) concept activation, during which they reject the unimportant parts of the document and divide content words into two groups "triggers" and "filters", 2) segmentation, during which they pick the words related to the words defined as "triggers" and decide which of those words should be skipped, and 3) identification of linguistic relations among the words, which helps to extract coherent phrases. Finally, the algorithm removes even more words, and what remains is the summary. While the description of the algorithm is sketchy, we can see that the algorithm runs over a full document and does not guarantee coverage of each sentence.

While these all are viable summarization approaches for many applications, none of these is suitable for generating summaries that satisfy the identified criteria. The major limitation of the reviewed approaches is that they all heavily rely on word frequency, which is a meaningful feature only when summarizing long documents. Unfortunately, the content of web pages is predominantly short. For instance, front pages of news sites typically have a number of short snippets each containing a headline followed by a few sentences from the linked articles; the articles themselves often contain only one long or a few short paragraphs.

In addition, most of the reviewed approaches do not offer any guarantees that keywords will be extracted from every sentence. Some of them do not preserve the original order of the words in the original text or extract separate words instead of word phrases.

In this paper, we propose a Skimming algorithm that satisfies the criteria outlined in the beginning of the section. We next present an overview of our approach.

## OVERVIEW OF OUR APPROACH

At a high level, our algorithm works as follows.

Firstly, every sentence is parsed to extract grammatical relations amongst its words. Secondly, a lexical tree based on these relations is constructed, where each node of the tree represents a word in the sentence. Thirdly, for every word in this tree, its grammatical (i.e., POS tags) as well as structural features (related to in-degree/out-degree, etc.) are extracted. These features are given to a trained classifier to determine whether or not to include the word in the skimming summary. Finally a subtree consisting of the selected words is constructed. This subtree represents the skimming summary that users interact with via an interface.

Observe that three key elements of our algorithm are a natural language parser, classifier, and a skimming interface. Below, we describe these three elements necessary for understanding of the Skimming algorithm, as well as its experimental evaluation.

### Stanford Parser

A preliminary user study with 20 participants who were blind [1], reported that screen-reader users were able to skim and comprehend text much better if the summary included connected word phrases, as opposed to disconnected word, e.g., "hands down" instead of "hands". To capture the relationships between words in a sentence, we used Stanford Parser [23], which is a probabilistic natural language parser that can analyze the grammatical and syntactical structure of sentences and extract relations.

The relations identified by the parser are very simple and, in the words of the developers, "quite accessible to non-linguists and effective in relation extraction applications". These typed dependencies are represented as relations between pairs of words (i.e., "John likes cake" the object of "John" is "cake"). Apart from the grammatical relations Stanford Parser also identifies the POS (Parts of Speech) of the words.

Stanford parser identifies a total of 48 different binary grammatical relations [27]. A grammatical relationship holds between a governor word and a dependent word. For example, the sentence *"Chewing gum can improve memory, say UK psychologists."* is processed into the following relations:

**Relation Name (Governor-Position → Dependent-Position)**
*Adjectival modifier (gum-2 → chewing-1);*
*Nominal subject (improve-4 → gum-2);*
*Auxiliary (improve-4 → can-3)*
*Nominal subject (say-6 → memory-5)*
*Clausal complement (improve-4 → say-6)*
*Noun compound modifier (psychologists-8 → uk-7)*
*Direct object (say-6 → psychologists-8)*

Using the grammatical relations generated by Stanford Parser a directed graph can be constructed, where the nodes are the words and the edges are the relations from the governor to the dependent (Figure 1). In the words of the developers, "the dependency graph is a tree (a singly rooted directed acyclic graph with no re-entrances)." [27]. Throughout the paper, we will treat the dependency graph as a tree and will refer to it as a *sentence tree*.

For training the classifier, we had to create labeled datasets. The details are described below.

### Dataset Generation for Training the Classifier

The quality of automated summarization is gradually approaching that of humans [31, 37]. However, in the previous section we explained why the existing automated summarization approaches are not suitable for skimming web pages. Part of the reason is that they have been trained on and fine-tuned for specific type of summaries, which are not suitable for skimming. Since we could not find an existing dataset of human-authored summaries that would be appropriate for skimming, we created our own.

To generate the gold-standard skimming summaries, we recruited 24 sighted participants. All of them were college-educated and were fluent in English. We collected 24 news articles on different topics with 5 paragraphs in each of the articles on average. The total size of the raw text included 674 sentences (17K words).

Each participant was asked to summarize 3 different articles, thus, generating 3 gold-standard summaries for each article. We then used a majority-vote approach to combine each of the 3 summaries into one, i.e., we chose only the words that were selected by at least 2 people, resulting in 43% compression ratio.

Since the purpose of skimming was to pick out the most salient information without rephrasing the original content, we stipulated that the subjects use extractive summarization. Because the point of skimming was to save time to the end users by presenting less information, but still have enough information to understand the content, we requested that the summary length be between 1/3 and 1/2 of the original content. However, we did not want to dictate how exactly to do summarization, so each participant was provided only with the following "loose" guidelines:

- Each sentence is to be summarized separately;
- The summary may only include words and phrases found in the summarized sentence;
- Word order and punctuation has to be preserved;
- Summaries should be as informative as possible.

### Interface

Summarization by itself does not satisfy the need for skimming; for screen-reader users, reading a summary is more like reading a separate, albeit shorter, narrative. Therefore, we designed an interface that allowed screen-reader users to switch seamlessly between reading the skimming summary and the original web page preserving the current reading position. This feature emulate the behavior of sighted readers who usually can scan quickly and then, at any point, slow down to read the text regularly. When switching from regular screen-reading to skimming, the reading position is set to the closest preceding word that was in both the original text and the summary.

The skimming interface was integrated into our Non-Visual screen-reading platform, which is a Java application that enables a screen reading interface for the Firefox web browser. It supports all the typical navigation shortcuts available in the mainstream screen-readers such as JAWS [21], e.g., paragraph / sentence / word / character navigation, pause / resume, etc.

A shortcut (i.e. Shift+S) lets the user toggle between skimming mode and full text mode. The skimming interface can be invoked on any web page content regardless of its length as long as the content is organized in sentences (e.g., snippets of news articles along with headlines), but lists of links that can be found in menus, on the other hand, should be read in their entirety.

## SKIMMING ALGORITHM

In this section, we describe the underpinnings of the proposed non-visual Skimming algorithm; specifically, we describe feature selection, training classifiers and the skimming algorithm.

### Feature Selection

The results obtained from the Stanford parser enabled us to identify a number of features that could be used for training a classifier. The features we chose were the Number of Outgoing Edges, Level in the Tree, Number of descendants, Incoming relation type, and POS tag (Figure 1). Each of the numerical features has been normalized with a scaling factor. The feature, Number of outgoing edges was normalized by the number of nodes in the tree. The feature, Level in the Tree was scaled by the highest level of a node in the tree. Finally, the feature, Number of descendants was normalized by the highest number of descendants a node can have in the tree.

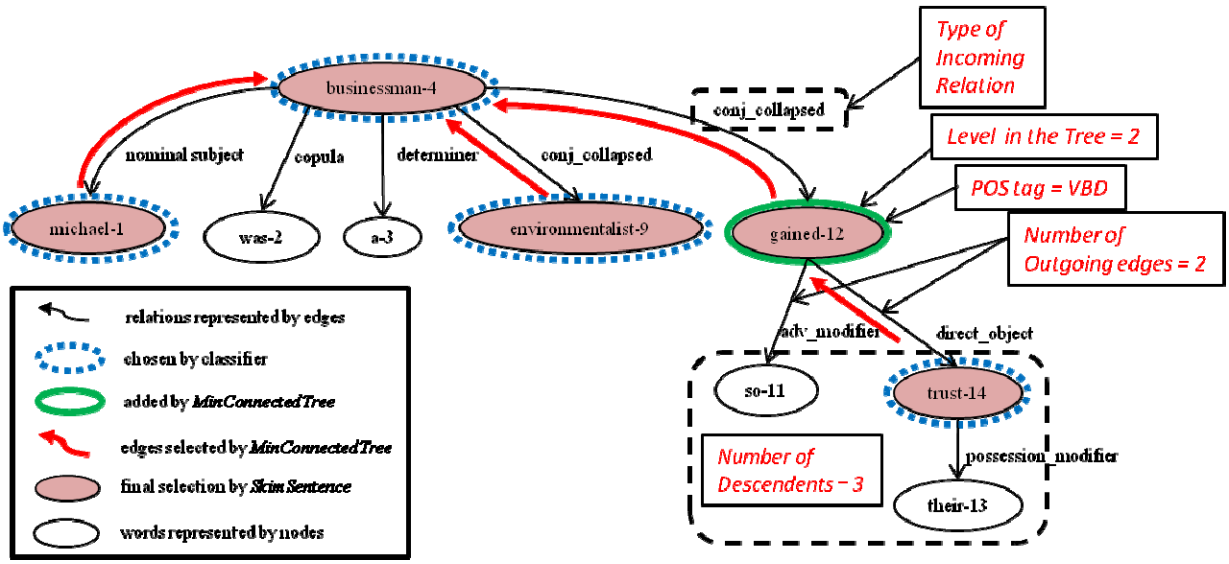



**Figure 1. Sentence tree and features for "*Michael was a businessman, as well as an environmentalist, and so gained their trust.*"**

### Training the Classifier

We used an open source machine learning library, called Weka [16], to train classifiers using various machine learning models. In order to generate training data points we need to generate feature vectors with their label/class. First, we used Stanford Parser [23] to generate feature vectors for each word in each sentence in our dataset. Then, to label/classify a feature vector, we check if the corresponding word appears in the gold summary; if it appears, we label the feature vector as "YES" class otherwise as "NO" class. "YES" class means this word is selected to be used in summary and "NO" class means otherwise. These datasets are used to train different classifiers (i.e., Linear SVM, Multilayer Perceptron, Random Forest) through the Weka machine learning library with 10-fold cross-validation; using 591 sentences (15300 words) for training and 83 sentences (1700 words) for testing.

The classifier-generated summary is formed by selecting words that were classified as "YES" and rearrange them by preserving the order in original sentence. To measure the accuracy of the classifier, we represented both the gold-standard summary and the classifier-generated summary as arrays of words. As many sentences had repeating words, it was necessary to align the words for more accurate measurements. Therefore, we ran the LCS (Longest Common Subsequence) algorithm to align the two arrays and compare the summaries.

We report the results in the first group in Table 1, under the caption "Output of the Classifiers alone". We found Linear SVM performs the best, therefore we decided to use it to classify words (*SkimSentence:Line 12*) in our proposed Skimming algorithm *SkimSentence* (next Section). In section after the next, we discuss this result in details and also show our algorithm *SkimSentence* improves the result.

### Skimming Algorithm

The classifier helped us pick up separate words in a sentence, however, the preliminary user studies [1] demonstrated that people tend to understand content better in the presence of phrases. Therefore, we designed the *SkimSentence* Algorithm to extract grammatically related important words of the sentence, while keeping comprehension level almost the same as our Gold-Standard summaries. The algorithm uses a trained classifier and another algorithm *MinConnectedTree* to skim the given sentence. The pseudocode for both algorithms is presented in this section.

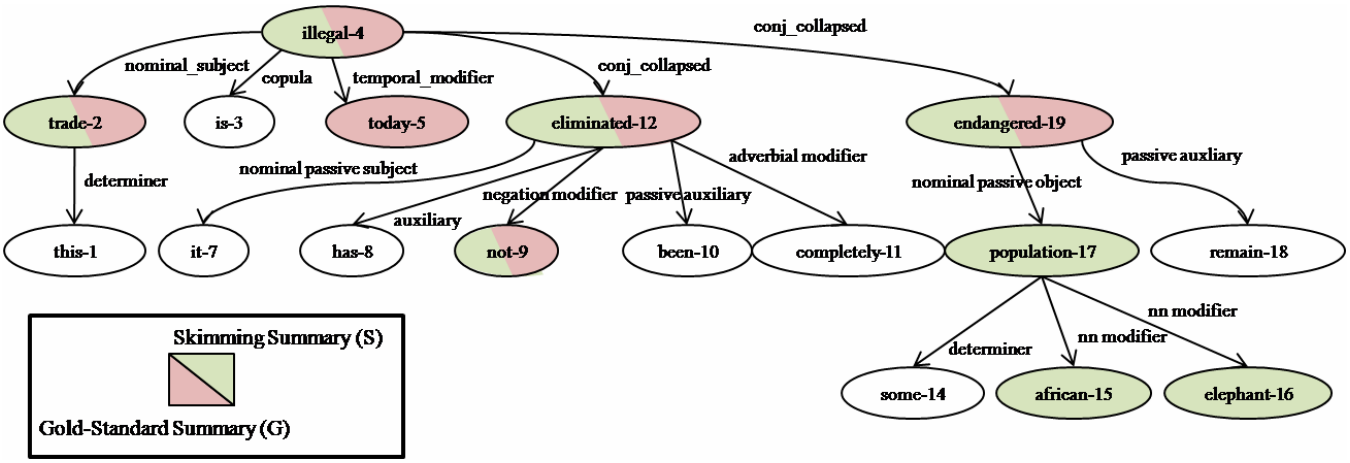
**Algorithm: *SkimSentence***

**R:** Set of Typed Dependency Relations $\{(W_{g1}, W_{d1}), (W_{g2}, W_{d2}), \ldots (W_{gn}, W_{dn})\}$ between words in a sentence, where Governor $W_{gi}$ depends on Dependent $W_{di}$ where $1 \leq i \leq n$.

**T:** Sentence Tree:(Parents:Child) ≡ (Governor : Dependent)

**Input**: S: sentence to be skimmed
**Output**: K: skimmed sentence

```
1. SN ← {} //initialize set of summary nodes
2. K ← [] //initialize list of words
3. R ← TypedDependencies(S) //stanford parser
4. T ← ConstructTree(R) //from typed dependencies
5. For each node N in T
6.     Do O ← #ofOutgoingEdges(N)
7.        L ← LevelInTree(N)
8.        C ← #ofDescendants(N)
9.        I ← IncomingRelationType(N)
10.       P ← POSTags(N)
11.       F ← <O,L,C, I, P> //feature vector
12.       Do If InSkimming(F)=True //SVM-classify
13.          Then SN.add(N)
14. MT ← MinConnectedTree(SN)
15. K ← OrderAndListWords(MT)
16. Return K
```




Skimming Summary (S): "*trade illegal, not eliminated, african elephant population endangered.*"
Gold-Standard Summary (G): "*trade illegal today not eliminated endangered*"
(F): *"This trade is illegal today, but it has not been completely eliminated and some African elephant population remain endangered."*

**Figure 2. Sentence tree illustration of the typical Skimming algorithm output compared to the human/gold summary**

**Algorithm: *MinConnectedTree***

**TN**: Temporary node
**Input**: S**:** Set of nodes of a sentence tree
**Output**: MS: Set of minimum connected nodes

```
1. MS ← {} //initialize set of nodes
2. For each node N in S
3.    Do TN ← N
4.       While TN ≠ NULL and TN.visited = False
5.          Do TN.visited ← true
6.             MS.add(TN)
7.             TN ← TN.Parent
8. Return MS
```

*SkimSentence* first uses the Stanford Parser to find all the typed dependencies / grammatical relations in the given sentence (*SkimSentence:Line 3*). Then it uses the list of grammatical relations to construct the sentence tree (*SkimSentence:Line 4*). "*Michael was a businessman, as well as an environmentalist, and so gained their trust.*"; the sentence tree corresponding to this sentence is illustrated by Figure 1 illustrates.

Once the tree is constructed, it creates a set of summary nodes which are chosen by our trained classifier (*SkimSentence:Line 5-13*). In order to choose the summary nodes, it first analyzes each of the nodes in the tree and identifies their features (*SkimSentence:Line 5-10*). #ofOutgoingEdges calculates the number of outgoing edges (normalized by total number of nodes) of a node (*SkimSentence:Line 6*). The method LevelInTree finds the level of a node in the Tree (normalized by the highest level of a node in the tree), where root is at level 1, children of the root is at level 2 and so on (*SkimSentence:Line 7*). #ofDescendents returns the total number of descendents (normalized by the highest number of descendents of a node in the tree) a node has (*SkimSentence:Line 8*).

The type of the relation between a node and its parent is determined by the method IncomingRelationType (*SkimSentence:Lines 9*). POSTags method finds the parts of speech of the word corresponding to a node (*SkimSentence:Line 10*). Going back to our example in Figure 1 let's focus on the node "gained-12". In the figure we can easily identify its features, number of outgoing edges is 2/9, level in the tree is 2/4, number of descendants is 3/8, type of incoming relation is conj_collapsed, POS tag is VBD (Verb, past tense).

With all the node features the algorithm constructs a feature vector F (*SkimSentence:Line 11*). In our example, the feature vector for the node "gained-12" is *<2/5, 2/4,3/9, conj_collapsed, VBD>*. Then the feature vector is fed to the trained SVM model in order to classify it to either "YES" or "NO". The InSkimming method takes a feature vector and returns True/False depending on whether the node is classified to "YES"/"NO" respectively, using the trained linear SVM classifier (*SkimSentence:Line 12*). If InSkimming returns True (which means it can be taken as part of skimmed sentence), the node is added to a set of summary nodes (*SkimSentence:Line 13*). In this example, the nodes "michael-1", "businessman-4", "environmentalist-9", and "trust-14" are chosen as summary nodes. In the figure these nodes are circled with dots (Figure 1).

The set of summary nodes constructed this way, does not guarantee if they have any grammatical relations between them. Moreover, adding more nodes which are grammatically related to these nodes will increase the comprehension level of the skimmed sentence. So to make sure that a minimum relation exists among these summary nodes, a second algorithm *MinConnectedTree* is used, to construct a subtree that connects all the summary nodes with minimum number of edges (*SkimSentence:Line 14*).

The algorithm *MinConnectedTree* constructs a tree with minimum number of edges necessary to cover a given set

of tree nodes. The algorithm runs two nested loops. The outer loop iterates over the given set of nodes (*MinConnectedTree:Line 2*). For each node it does a traversal towards root of the tree (*MinConnectedTree:Line 4-7*). In the inner loop, while traversing towards the root, it checks if the node has already been visited or not. If the node has not been visited, it is marked as visited and is added to a set of output nodes (*MinConnectedTree:Line 6*). After the nested loops are executed, a set of nodes are returned, which forms a tree with minimum number of edges (*MinConnectedTree:Line 8*). In Figure 1, from each of the summary nodes (circled with dots), the algorithm traversed towards the root and picked up any non summary nodes on the way. The traversal paths have been marked with bold lines with arrowhead in the figure. The only non-summary node that has been picked up (apart from the summary nodes) by *MinConnectedTree*, is "gained-12". All the nodes that have been selected by *MinConnectedTree* are shaded in the figure.

In the *SkimSentence* algorithm, once the tree with minimum number of edges is constructed (*SkimSentence:Line 14*), a skimmed sentence is made by ordering the nodes according to the position of the corresponding words in the original sentence, using the method OrderAndListWords (*SkimSentence:Line 15*)

Next we present the outcomes of our algorithm on a sample sentence. In Figure 2, we present the corresponding sentence tree and the gold-standard and skimming summaries. For better understanding we color the nodes. The words that appear in the Gold-standard summary are colored with dark red and the words chosen by our *SkimSentence* algorithm are colored with light green. If a word is chosen by our algorithm as well as human/gold skimming, half of the node is colored with dark red and other half is colored with light green.

## EXPERIMENTS WITH THE DATASET

### Experimental Setup

To evaluate our skimming algorithm, we did experiments on our dataset. In this section we do a comparative study between the "summaries generated by Classifiers alone" and the "summaries generated by *SkimSentence*".

We report the results using the standard metrics of precision, recall, and F-measure, as well as the compression ratio. We define *precision* as the ratio of the number of the words appearing in both arrays to the size of our skimmed summary. We define *recall* as ratio of the number of the words appearing in both arrays to the size of the gold-standard summary. *F-measure* is the harmonic mean of recall and precision. Finally, we define the *compression ratio* as the ratio of the number of words in a skimmed summary to the number of words in the original sentence.

| | Random Forest | Multi-Layer Perceptron | Linear SVM |
|---|---|---|---|
| **Output of the Classifiers alone** | | | |
| **Precision** | 58.92% | 59.93% | 59.27% |
| **Recall** | 54.01% | 66.75% | 68.63% |
| **F-measure** | 0.56 | 0.63 | 0.64 |
| **Compression** | 33.62% | 41.16% | 43.20% |
| **Output of the algorithm *SkimSentence*** | | | |
| **Precision** | 57.36% | 58.34% | 63.40% |
| **Recall** | 63.77% | 75.78% | 76.57% |
| **F-measure** | 0.60 | 0.65 | 0.69 |
| **Compression** | 40.95% | 48.34% | 46.81% |

**Table 1 – Output of three different classifiers**

### Results and Discussion

Although we tried all of the models available in Weka, in Table 1 we report only on the top three classifiers: Random forest, Multi-layer perceptron, and linear SVM-based model (implemented in the lib-svm [11] software). Of the three classifiers, we chose linear SVM for further experiments as it had the best classification results on our dataset. The comparative performance of different classifiers on our dataset was not in the scope of this paper, especially, because classifier performance often varies depending on the size of the training dataset [30, 34].

The SVM classifier yielded the best combination of the precision: 59.2%, recall: 68.6%, F-measure: 0.64 and, the compression ratio: 43.2%. By running the Skimming algorithm on the classifier results, we improved the precision to 63.4%, recall to 76.5%, and F-measure to 0.69, at the cost of the compression of 46.8%. The 4% improvement of precision and 8% improvement of recall, at the 4% difference in compression justifies the use of the Skimming algorithm on top of the classification results.

While these measures give the overall idea of the algorithm's accuracy, **it should be noted that, for the purposes of skimming, we favored recall over precision**, because the correctly recalled words typically carry the most meaning. At the same time, we balanced recall with precision not to introduce too much "noise"; **it is also notable that the "noise" words that were "erroneously" chosen by the algorithm could carry just as much meaning as the words in the gold-standard summary**, because while shortening a sentence, a human may choose just one of any two equally meaningful words. This observation is supported by the user study that showed no statistically significant difference in the performance of subjects over the gold-standard and skimmed summaries. Finally, in our experiments, we also balanced the compression ratio, so that the skimmed summary was no longer than the gold-standard one.

## USER STUDY

### Participants

The Disability Resource Center at the Arizona State University helped us recruit 23 participants who are blind to take part in the user study. Gender representation was 7 male and 16 female. The ages of the participants were evenly distributed and varied from early twenties to late sixties. Three of the subjects considered themselves expert computer users, fourteen of them were very comfortable, five were comfortable and one was mildly comfortable with computers. All subjects were well-versed in the use of screen readers, with JAWS as their primary screen reader. Internet usage was more than 20 hours per week for twelve subjects, 11-20 hours per week for eight subjects, 6-10 hours per week for two subject, and 1-5 hours a week for only one subject.

### Experimental Set Up

To evaluate our approach, we built the proposed Skimming interface. The articles chosen for the study were typical web news articles. However, they did not contain any links or images to reduce the variation among the articles. We used RealSpeak Samantha voice [32] with the speech rate of 140 words per minute.

To evaluate the interface and the underlying algorithm, the subjects were asked to perform three tasks in each of the listening-and-comprehension and searching scenarios. In each of the three tasks the subjects were either using the Gold-standard summary, Skimming summary, or the original text of the articles.

### Listening-and-Comprehension Scenario

In a within-subjects experiment that had a 3 by 3 cell design, we had each subject listen to 3 different articles using 3 types of skimming: Gold-standard summary G, Skimming summary S, and the full text F (Table 2). We varied the order of the tasks for counterbalancing and to avoid bias. Before the experiment, the subjects were given a sample listening-and-comprehension task to practice with. They were not allowed to use navigation shortcuts in this task. Each task took approximately 10 minutes, which included the question answering section as well.

A set of 12 questions was asked to test the overall comprehension and retention of information. The first question was open-ended and the rest were multiple-choice. The questions we formulated to cover the major points of each article without regard for which content actually appeared in the corresponding gold summary. The questions covered several parts of speech that carried information: 1 question was on the article topic (i.e. "What is the article about?" – "African Elephants"), 4 questions on the nouns (e.g., "In search of what do the Mali elephants migrate?" – "Water"), 3 questions on the verbs (e.g., "According to the text, African Elephants cannot be easily" – "Domesticated"), 2 on the numeric values (e.g., "How much does an elephant weigh at birth?" – "200 pounds"), and 2 on the adjectives/adverbs (e.g., "Mali elephants are described as?" – "nomadic"). We approximately followed the distribution of the parts of speech in the original articles and were consistent in this break-down across all articles.

| **S: Skimming by SkimSentence** |
|---|
| *Elephants cows live family herds young, adult males bulls tend roam. Having baby elephant commitment. Elephants have longer pregnancy mammal 22 months. Cows give birth to one calf two to four years. Birth, elephants weigh 200 pounds stand 3 feet tall. African elephants, unlike asian relatives, are not domesticated. Range africa rain forests central west africa. Elephants found desert. Herd mali elephants migrates route desert search water.* |
| **G: Gold/Human Skimming** |
| *Female cows live family herds with young males bulls roam own. Having baby serious commitment. Longer pregnancy 22 months. Birth to calf every two four years. At birth weigh 200 pounds 3 feet tall. African elephants not easily domesticated. Range sub-saharan africa and central west. Northernmost elephants found in malis sahel desert. Herd of mali elephants migrates in circular route search water.* |
| **F: Original paragraph:** |
| *Female elephants cows live in family herds with their young, but adult males bulls tend to roam on their own. Having a baby elephant is a serious commitment. Elephants have a longer pregnancy than any other mammal almost 22 months. Cows usually give birth to one calf every two to four years. At birth, elephants already weigh some 200 pounds (91 kilograms) and stand about 3 feet (1 meter) tall. African elephants, unlike their Asian relatives, are not easily domesticated. They range throughout sub-Saharan Africa and the rain forests of central and West Africa. The continent's northernmost elephants are found in Malis Sahel desert. The small, nomadic herd of Mali elephants migrates in a circular route through the desert in search of water* |

**Table 2. Skimming and Gold-standard summaries**

### Searching Scenario

In a within-subjects experiment that had a 3 by 3 cell design, we had each subject skim through 3 different articles using the Gold-standard summary G, Skimming summary S, and the full text F. The conditions were counterbalanced across subjects to avoid bias. Before each task, the subjects were asked a question, the answer to which they had to find in the article. The subjects were instructed not to try keyword searching because some would and some would not depending on their experience with screen readers. However, to make the scenario realistic, the questions were formulated using wording different from those that appears in the actual article, so that one could not easily search for the exact keywords, e.g., if the original sentence was "*Cows usually give birth to one calf every two to four years.*" The question was: "How frequently do female elephants have calves?"

The answer to the question was contained in the 4th and the last paragraph of each article to increase the time required to find the answer. Having the answer in the beginning of the articles would have caused the users to find the answer equally fast with and without skimming, defeating the purpose of the experiment. Varying the location of the

answer would have increased the variation of the completion times.

Although the answers in the search scenarios were located in the last paragraph, the subjects would not have been able to notice the pattern because there were only 3 tasks to learn from and the length of passages varied (summary is shorter). The task order was randomized.

In this task, we measured the time it took subjects to answer the question. We also measured the number of keystrokes the subjects pressed to find the answer. The subjects were given a sample task before the experiment and 5 minutes to practice with the skimming interface. They were allowed to use any navigation shortcuts they were used to. Each task took approximately 10 minutes.

### Questionnaires

Following each task in either of the scenarios, the participants were asked to rate the perceived difficulty of the task on a 5-point Likert scale (1=Very Easy to 5=Very Hard, Figure 3b, Figure 5b). At the end of the experiments, we read statements about the skimming experience to the subjects and asked them to rate the statements on a 5-point Likert scale (1=Strongly Disagree to 5=Strongly Agree) – Table 3 summarizes the ratings. Finally, we also asked general questions about subjects' computer use, age, comfort level, onset of blindness, etc. We used these questions to test for performance differences among groups of subjects.

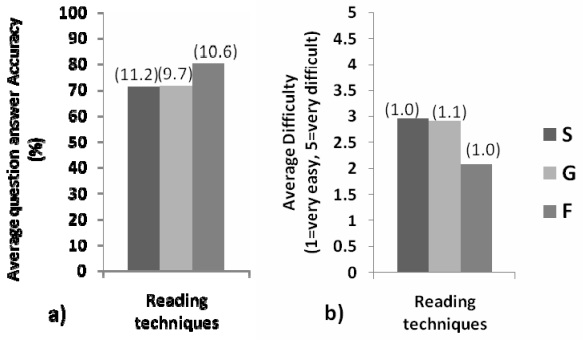



**Figure 3. a) Average accuracy and b) Average difficulty (St. Dev.) of question answering with S, G and F**

### Hypotheses Testing

We formulated a series of specific hypotheses that we accepted or rejected based on the statistical significance testing. In this section, we provide the details on the hypothesis and discuss the results. Finally, we present the results of the analysis of the post-completion questionnaire. *We used a t-test for statistical significance testing with the significance level of alpha = 0.01.* The following are the specific hypotheses tested in the experiments.

**H1:** After listening to summaries S and G, subjects can answer reading-and-comprehension questions equally well.

**Results:** Subjects were able to answer the reading-and-comprehension questions correctly with accuracy of 71.01% (St. Dev.=11.19%) for summary S and 71.37% (St.Dev.=9.67%) for summary G (Figure 3a). Using the one-tailed paired t-test, we found that there is no statistically significant difference between the question answering accuracy for summary S and G (t=-0.134, df=22, p=0.894), accepting H1.

We found no statistically significant differences for specific types of questions between summaries S and G (Figure 4).

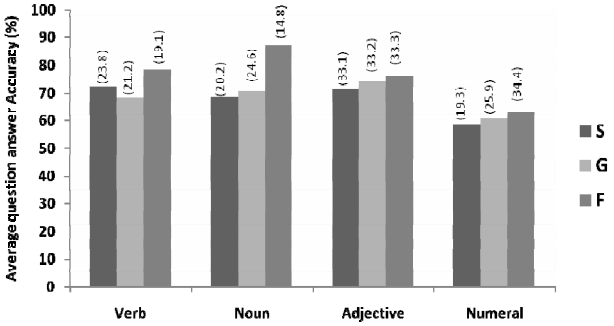



**Figure 4. Accuracy of different question types**

**Discussion:** One of the most important findings of these experiments was that the subjects were able to answer accurately about 70% of the questions about the text after listening to either skimming summary S or the gold-standard summary G.

**H2:** After listening to summary S, G & full text F, subjects can answer reading-and-comprehension equally well.

**Results:** Subjects were able to answer 71.01% (St. Dev.=11.19%) of the reading-and-comprehension questions correctly for summary S, 71.36% (St. Dev.=9.67%) for summary G and 80.07% (St. Dev.=10.58%) for full text F (Figure 3a). A one-way ANOVA was used to test accuracy differences in answering three types of summaries S, G, and F. The accuracy with which subjects gave answers differed significantly across the three types of summaries, $F(2, 66) = 5.48$, $p = 0.0063$. The post-hoc test for linear trend also shows that there is a trend, that answering accuracies increases as you read summary type S, G and F respectively, (Slope 0.04529, $p = 0.0047$), rejecting H2.

**Discussion:** This result was as expected, because, after all, full text was easier to understand and contained all the answers to the questions. It is notable that while the length of text shortened by more than 50%, the performance was not compromised significantly (at most 10%). We believe that the subjects could not answer 100% of questions after reading the full text due to information overload.

**H3:** Subjects perceive question-answering tasks to be of the same difficulty after listening to a summary G/S and full text F.

**Results:** After the completion of each listening-and-comprehension task, we asked the subject to rate their difficulty on a Likert scale (very easy=1 and very difficult=5). Subjects rated the difficulty level of the task with summary S as 2.95 on average (St. Dev.=1.02), rated summary G as 2.92 on average (St. Dev.=1.16), and rated

the full text F as 2.08 on average (St. Dev.=1.06). A one-way ANOVA was used to test the difficulty in answering three types of summaries S, G, and F. The difficulty with which subjects gave answers did not differ significantly across the three types of summaries, F (2, 66) = 2.501, p = 0.0898, thus we accept H3. (Figure 3b)

**Discussion:** Reading full text is supposed to be easier than reading a skimmed summary. However, reading the full text may be putting a higher cognitive load on the subject resulting in information overload, and making it harder to remember information and answer questions. The fact that subjects rated both G and S summaries at the same level of difficulty suggests that our Skimming summary is very close to human-generated summaries.

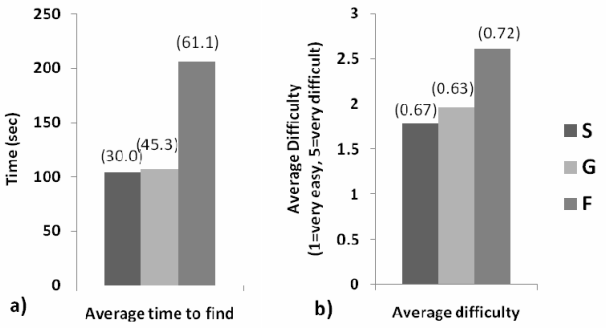



**Figure 5. a) Average time and b) Average difficulty in the searching task using skimming vs. regular shortcuts**

**H4:** Subjects could find information faster while skimming with G/S, than when using screen-reader navigation shortcuts.

**Results:** While using skimming to search for an answer to a question, the subjects took on average 104.13 seconds (St. Dev. 30.00) with S and 107.08 seconds (St. Dev. = 45.33) with G. Using regular screen-reader navigation shortcuts, it took subjects 205.78 seconds (St. Dev. = 61.17) to answer the questions (Figure 5a). A one-way ANOVA was used to test the differences among times subjects took while searching for the answer to a question, with S, G and using regular screen reader shortcuts. The time they took differed significantly across three techniques, F (2, 66) = 35.02, p < 0.0001. The post-hoc test using Tukey's multiple comparison test shows the difference in time for S and F was significantly different (mean diff = -100.60, q=10.41). Same goes for G and F as well (mean diff = -97.35, q=10.08); thus, we accept H4.

**Discussion:** Subjects were able to find the answer almost twice as fast using skimming, compared to using regular screen-reader navigation shortcuts. This result demonstrates that skimming can save screen-reader users a substantial amount of time, depending on the size of the text. In our case, since each article had 5 paragraphs, the subjects saved up to 30 seconds per paragraph while searching. The fact that there was no difference between S and G again speaks in favor of our algorithm.

**H5:** Subjects perceived ad-hoc searching to be more difficult compared to using skimming S or G.

**Result:** In the search task subjects perceived a difficulty level of 2.60 (St. Dev. = .72) on average while using screen-reading shortcuts without skimming. With skimming S, subjects perceived a difficulty of 1.78 (St. Dev. = 0.67) and with skimming G 1.95 (St. Dev. = 0.63) on average (Figure 5b). Conducting one-way ANOVA test shows significant differences in the perceived difficulty level while finding answers with skimming S, G and using regular screen-reader shortcuts, F (2, 66) = 9.487, p = 0.0002. The post-hoc test for linear trend also shows: the perceived difficulty increases as you use summary type S, G and screen-reader shortcuts respectively, (Slope 0.4130, p = 0.0001). Another post-hoc test, Tukey's multiple comparison test shows that using screen-reader shortcuts is significantly more difficult (mean diff = -0.83, q = 5.84) than skimming S. Similarly using screen-reader shortcuts were more difficult than using skimming G (mean diff = -0.65, q = 4.61); so we accept H5.

**Discussion:** As the search task using screen-reader shortcuts was involved with listening to full sentences instead of summaries, subjects had to put effort and time to listen in full and find the answer to the question. So using screen-reader shortcuts turns out to be difficult.

**H6:** The number of keystrokes pressed was the same between searching with skimming and searching with navigation shortcuts.

**Result:** The average number of keystrokes pressed was 36.60 (St. Dev = 16.21) for Skimming summary S, 35.65 ( St. Dev. = 15.85) for Gold Summary G, and 37.08 (St. Dev. = 26.61) for Full text F. A one way ANOVA test showed no statistically significant differences between the number of keystrokes while searching with S, G and using navigation shortcuts F (2, 66) = 0.0301, p = 0.9703. Thus, we accept H6.

**Discussion:** These findings demonstrate that skimming does not require more effort in terms of pressing shortcuts, compared to the regular navigation. At the same time, skimming increases the navigation speed twofold (H4).

**H7:** Subjects were not able to distinguish between human- and computer-generated summaries.

**Results:** After each listening-and-comprehension, subjects were asked to identify whether the text was written by a human. On average in 66.66% of cases (St. Dev.=48.3%) cases they mistook Skimming Summary S for the Gold-standard summary G. And, on average, in 76.2% of cases (St. Dev. 43.6%) cases they were not able to distinguish between Skimming Summary S and Gold Summary G. The one-tailed paired t-test shows that there is no significant difference between the choices subject made while detecting S and G as human written or not (t=1.369, df=20, p=0.186). This means most of the time they were

not able to distinguish between Gold-standard summary and Skimming summary; confirming H7.

**Discussion:** This is another result that supports the claim that the proposed Skimming algorithm could generate summaries that were almost as good as those written by humans.

We observe no difference in performance on these tasks between groups of subjects:

**H8:** who lost their eyesight before the age of 10, 15, or 20 and those who lost sight later.

**H9:** belonging to different age categories.

**H10:** with different comfort level and different amount of time spent with computers.

### Post-Completion Questionnaire Analysis

The post completion questions and answers about the skimming experience are presented in Table 3. The analysis showed that the standard deviation for all questions was within one point, demonstrating high consistency.

| General Statements | Avg. (St. Dev.) |
|---|---|
| I wish I could look through articles faster, than I can with a screen reader | 4.30 (0.82) |
| I often experience difficulties looking for desired information within an article | 3.60 (1.11) |
| Skimming made reading through articles faster | 4.22 (0.85) |
| I understood articles equally well when skimming and when reading the full article | 2.82 (1.07) |
| I want to use skimming in the future | 4.26 (1.00) |
| I would like to see skimming implemented in a commercial screen reader | 4.60 (1.15) |

**Table 3 – Average 5-Point Likert scale values (St. Dev.) (Scale 1=Strongly Disagree to 5=Strongly Agree)**

From the statements made by subjects, we can conclude that skimming has a lot of potential to improve the reading experience for screen-reader users. Although the subjects thought that their comprehension was somewhat impeded in the skimming mode, almost all subjects agreed that skimming made the reading faster and they wanted to use skimming in the future.

One of the subjects made a comment that non-visual skimming is "…very useful to glimpse over articles. May be very useful for test takers and also for sighted people (while driving and reading news/articles)". It indicates a possible application of skimming that would allow not only blind but also sighted users to take advantage of audio skimming. Skimming can be useful to sighted users who want to relax their eye and listen to web content faster.

## TESTIMONIALS

In this section, we present a short selection of the positive comments made by subjects about skimming:

"*It's less labor intensive, I set to skim mode and sit and listen as opposed to reading full sentences. I am more inclined to use skimming now.*"

*"…easier because it was faster. I usually go line by line. I miss important things, when I go line by line. And sometimes I have to go back and start from beginning to double check. But skimming picks the important thing for me"*

*"it makes efficient and quick access to the document. I can get through a longer doc speedily. I would like to buy this tool. I think this can also be helpful to skim through ads"*

Although the majority of comments were highly positive, 3 participants were not as happy with the skimming interface:

*"I like detailed information much more."*

*"People who are not born blind, they can perceive skimming fast. But people who are born blind, it's new to them, needs training."*

*"I prefer normal reading. I don't like fragmented words."*

While it is true that non-visual skimming needs some practice (just like visual skimming), it is notable that the subjects who made those comments did not perform any worse than the rest of the participants; they completed the searching task faster with skimming compared to ad-hoc searching, and they answered most of the questions in the listening and comprehension tasks.


## ACKNOWLEDGEMENT

Research reported in this paper was supported by NSF Awards IIS-0808678 and CNS-0751083.


## CONCLUSIONS AND FUTURE WORK

The approach proposed in this paper can summarize one sentence at a time without the dependence on the entire document. It can extract meaningful word combinations from most sentences and preserve the original order of the extracted text. All these properties make the proposed summarization approach more suitable for supporting a non-visual skimming interface. Our experiments give reasons to believe that non-visual skimming can enable screen-reader users to browse textual content faster.

This work opens the possibilities to further accelerate non-visual skimming by varying the compression ratio of summarization. This direction of research will require more experiments to evaluate how much information can be retained as we further compress the original text. Armed with the ability to change the amount of information as necessary, people who are blind may be able to battle the information overload more effectively and enjoy the speed-reading commonly used by sighted readers.

There is also room for improving the accuracy of summarization. For instance, we could evaluate more machine learning models. However, as we demonstrated in our user studies, machine-generated summaries do not necessarily have to match those generated by humans to be of equal utility.

Finally, because webpage articles often have formatting, the skimming algorithm can also be extended to take into account the formatting used to emphasize text.

## REFERENCES


[1] Ahmed, F., Y. Borodin, Y. Puzis, and I.V. Ramakrishnan, *Why Read if You Can Skim: Towards Enabling Faster Screen Reading*, in *W4A*. 2012.

[2] Ahmed, F., M.A. Islam, Y. Borodin, and I.V. Ramakrishnan, *Assistive web browsing with touch interfaces*, in *Proceedings ACM SIGACCESS*. 2010, ACM: Florida, USA. p. 235-236.

[3] Amitay, E. and C. Paris., *Automatically summarising web sites: is there a way around it?*, in *CIKM*. 2000, ACM. p. 173.

[4] Berger, A.L. and V.O. Mittal., *Ocelot: a system for summarizing web pages*, in *SIGIR*. 2000, ACM Press. p. 144.

[5] Berghel, H., *Cyberspace 2000: Dealing with Information Overload.* COMMUNICATIONS OF THE ACM, 1997. 40-2.

[6] Bigham, J.P., A.C. Cavender, J.T. Brudvik, J.O. Wobbrock, and R.E. Ladner, *WebinSitu: A Comparative Analysis of Blind and Sighted Browsing Behavior*, in *Assets*. 2007, ACM New York, NY, USA ©2007: Tempe, Arizona, USA.

[7] Borodin, Y., F. Ahmed, M.A. Islam, Y. Puzis, V. Melnyk, S. Feng, I.V. Ramakrishnan, and G. Dausch, *Hearsay: a new generation context-driven multi-modal assistive web browser*, in *WWW*. 2010, ACM: Raleigh, North Carolina, USA.

[8] Borodin, Y., J.P. Bigham, G. Dausch, and I.V. Ramakrishnan, *More than meets the eye: a survey of screen-reader browsing strategies*, in *Proceedings of W4A*. 2010, ACM: Raleigh, North Carolina. p. 1-10.

[9] Borodin, Y., J.P. Bigham, R. Raman, and I.V. Ramakrishnan, *What's new?: making web page updates accessible*, in *Proceedings of ACM SIGACCESS*. 2008, ACM, Canada.

[10] Borodin, Y., J. Mahmud, I.V. Ramakrishnan, and A. Stent, *The HearSay non-visual web browser*, in *Proceedings of W4A*. 2007, ACM: Banff, Canada.

[11] Chang, C.-C. and C.-J. Lin, *LIBSVM : a library for support vector machines*, in *ACM Transactions on Intelligent Systems and Technology*. 2011. p. 27:1-27:27.

[12] Chuang, W.T. and J. Yang, *Extracting sentence segments for text summarization: a machine learning approach*, in *Proceedings of ACM SIGIR*. 2000, ACM: Greece. p. 152-159.

[13] DeJong, G., *Skimming stories in real time: An experiment in integrated understanding*, in *Research Report 158*. 1979, Department of Computer Science, Yale Univesity.

[14] Fischer, G. and C. Stevens., *Information access in complex, poorly structured information spaces*, in *CHI*. 1991, ACM Press. p. 63–70.

[15] Goldstein, J., M. Kantrowitz, V.O. Mittal, and J.G. Carbonell, *Summarizing Text Documents: Sentence Selection and Evaluation Metrics*, in *In Research and Development in Information Retrieval*. 1999. p. 121-128.

[16] Hall, M., E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I.H. Witten, *The WEKA Data Mining Software: An Update*, in *SIGKDD Explorations*. 2009.

[17] Harper, S. and N. Patel, *Gist summaries for visually impaired surfers*, in *Proceedings of ACM SIGACCESS*. 2005, ACM: Baltimore, MD, USA.

[18] Hori, C., S. Furui, R. Malkin, H. Yu, and A. Waibel, *A statistical approach to automatic speech summarization.* EURASIP J. Appl. Signal Process., 2003. p. 128-139.

[19] Islam, A.M., Y. Borodin, and I.V. Ramakrishnan, *Mixture Model based Label Association Techniques for Web Accessibility*, in *UIST*. 2010: New York, NY, USA.

[20] Jacobs, P.S., *To parse or not to parse: relation-driven text skimming*, in *Proceedings of ACL- Volume 2*. 1990, Association for Computational Linguistics: Finland. p. 194.

[21] JAWS, *Screen reader from Freedom Scientific*. 2011. http://www.freedomscientific.com/products/fs/jaws-product-page.asp.

[22] JAWS_Skimming. *Skim Reading*. 2011; Available from: http://www.freedomscientific.com/Training/Surfs-up/Skim_Reading.htm.

[23] Klein, D. and C.D. Manning., *Accurate Unlexicalized Parsing*, in *Proceedings of ACL*. 2003. p. 423-430.

[24] Lamb, A. and L. Johnson. *Skimming and Scanning*. 1999; Available from: http://42explore.com/skim.htm.

[25] Litvak, M. and M. Last, *Graph-based keyword extraction for single-document summarization*, in *Proceedings of the Workshop on Multi-source Multilingual Information Extraction and Summarization*. 2008, Association for Computational Linguistics: United Kingdom. p. 17.

[26] Mahmud, J.U., Y. Borodin, and I.V. Ramakrishnan, *CSurf: a context-driven non-visual web-browser*, in *Proceedings of WWW*. 2007, ACM: Banff, Alberta, Canada.

[27] Marneffe, M.-C.d., B. MacCartney, and C.D. Manning, *Generating Typed Dependency Parses from Phrase Structure Parses.*, in *LREC* 2006.

[28] MindTools. *Speed Reading*. Available from: http://www.mindtools.com/speedrd.html.

[29] NVDA. *NonVisual Desktop Access*. 2009 [cited 2009]; Available from: http://www.nvda-project.org/.

[30] Popovici, V., W. Chen, B.G. Gallas, C. Hatzis, W. Shi, F.W. Samuelson, Y. Nikolsky, M. Tsyganova, A. Ishkin, T. Nikolskaya, K.R. Hess, V. Valero, D. Booser, M. Delorenzi, G.N. Hortobagyi, L. Shi, W.F. Symmans, and L. Pusztai, *Effect of training-sample size and classification difficulty on the accuracy of genomic predictors.* Breast Cancer Research, 2010.

[31] Radev, D.R., E. Hovy, and K. McKeown, *Introduction to the special issue on summarization.* CL, 2002. 28(4): p. 399-408.

[32] RealSpeak. *RealSpeak Solo Direct Voices for Freedom Scientific Products*. 2011; Available from: http://www.freedomscientific.com/downloads/RealSpeak-Solo-Direct-Voices/RealSpeak-Solo-Direct-Downloads.asp.

[33] Sweller, J., *Cognitive load during problem solving: Effects on learning*, in *Cognitive Science*. 1988.

[34] Wharton, S.W., *An analysis of the effects of sample size on classification performance of a histogram based cluster analysis procedure,* in *Pattern Recognition*. 1984. p. 239-244.

[35] Window-Eyes, *Screen Reader GW Micro*. 2010. http://www.gwmicro.com/Window-Eyes.

[36] Zajicek, M., C. Powell, and C. Reeves, *A Web navigation tool for the blind*, in *Proceedings of the third international ACM conference on Assistive technologies*. 1998, ACM: Marina del Rey, California, United States. p. 204-206.

[37] Zechner, K., *Fast Generation of Abstracts from General Domain Text Corpora by Extracting Relevant Sentences*, in *COLING '96 Proceedings of the 16th conference on Computational linguistics - Volume 2*. 1996.